

CSC 4575: Capstone Report

Daniel Craig

April 2026

1 Executive Summary

The secure hybrid pipeline engineered for this project implements a defense-in-depth strategy to secure communication between the client and server. This pipeline ensures that the data is encrypted and that data from other parties is verified before sensitive information is exchanged.

Cryptographic Primitives:

- **Bulk Data Transfer: AES-GCM (256 bit)** is used to provide high-speed symmetric encryption for the message payload.
- **Asymmetric Key Exchange: X25519 (Elliptic Curve Diffie-Hellman)** is used to securely create a shared session between Alice and Bob without sending the key itself over the network.
- **Identity Authentication: RSA-2048** digital signatures (using PSS padding and SHA-256) are used to verify the identities of both the client and server. It binds their long-term identity keys to the ephemeral session keys.

Satisfaction of Foundational Goals

- **Confidentiality:** Achieved through AES-GCM encryption, which ensures that the plaintext is transformed into ciphertext that is undetectable to unauthorized observers.
- **Integrity:** Maintained through the AES-GCM authentication tag, which detects any modifications to the ciphertext.
- **Authenticity:** Verified by the RSA signatures, ensuring that the ephemeral session keys are bound to the verified identities of the server and client. This helps prevent Man-in-the-Middle (MitM) attacks.

2 Protocol Architecture & Handshake Flow

Sequence Breakdown:

1. **Ephemeral Key Generation:** Both the client (Alice) and the server (Bob) generate a new X25519 private key and derive 32-byte public keys.
2. **Alice's Handshake Initiation** Alice signs her 32-byte X25519 public key using her long-term RSA-2048 private key.
3. **Client-to-Server Transmission:** Alice sends a 288-byte packet containing her 256-byte RSA signature followed by her 32-byte X25519 public key.
4. **Server Verification & Response:** Bob receives the data and uses Alice's RSA-2048 public identity key to verify the signature against the received X25519 public bytes. If the key is validated, Bob signs his own X25519 public key with his RSA-2048 private key and sends the 288-byte package back to Alice.
5. **Client Verification:** Alice verifies Bob's signature using his RSA-2048 public key.

6. **Key Derivation:** Once both signatures are verified, both Alice and Bob compute the shared secret using the X25519 points to derive the final AES key via HDKF

Key Delineation:

- **Long-Term Identity Keys (RSA-2048):** These are static keys that represent the "identity" of the users. The private key is kept secret and used to sign the handshake, while the public key is shared to verify signatures. These keys remain the same accross multiple sessions.
- **Ephemeral Keys (X25519):** These are temporary keys generated for a single connection. They are used for the Diffie-Hellman exchange to create a session key and are destroyed once the connection is closed.

Design Defense:

The RSA signature must be applied to the X25519 public bytes before transmission to provide Entity Authentication. Mathematically, this binds the unauthenticated Diffie-Hellman exchange to a verified long-term identity.

This sequence prevents a Man-in-the-Middle attack. Without the signature, a hacker could intercept the X25519 public keys and subsitute their own. By signing the public key, Alice proves that the specific ephemeral session key was generated by the owner of the verified RSA identity key. This makes it impossible to make an unauthorized substitution.

3 Network Verification

- **Proof of Confidentiality:**

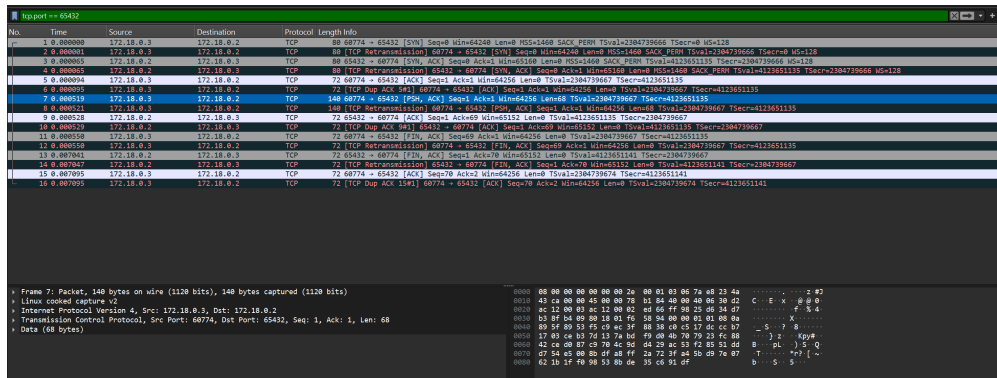


Figure 1: Week 1 AES-GCM Encrypted Payload (68 bytes)

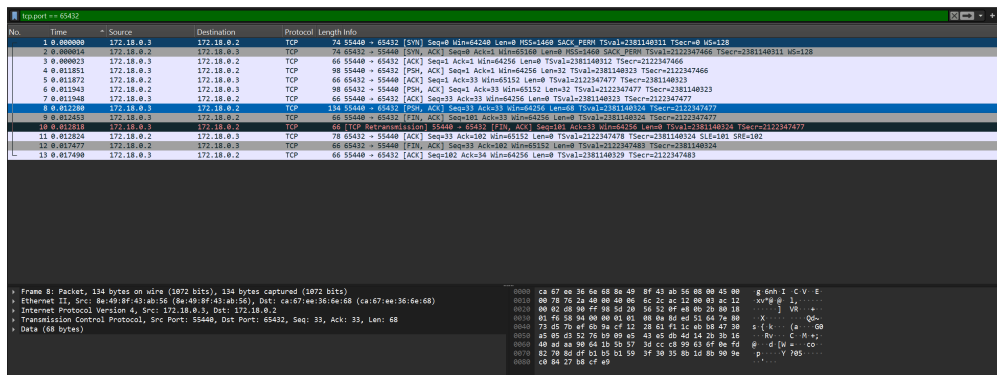


Figure 2: Week 2 AES-GCM Encrypted Payload (68 bytes)

Figure 1 & 2 shows the high-entropy ciphertext. The preview on the bottom right hand side shows the ASCII preview which is only random characters. Proving that the plaintext message is completely confidential from network observers.

- **Proof of Key Exchange:**

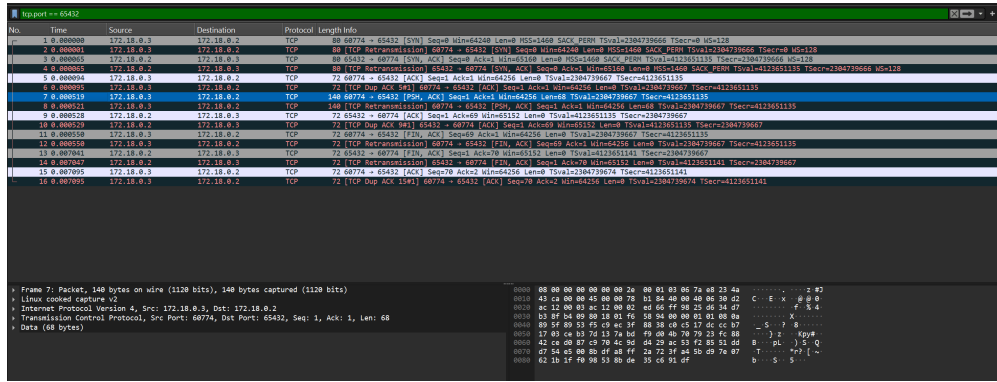


Figure 3: Week 1 Proof of Out-of-Band Key Exchange (Pre-Shared Key)

Figure 3 shows how the connection moves from the handshake to encrypted data. The week 1 code uses a pre-shared key which doesn't show within the tcp exchange.

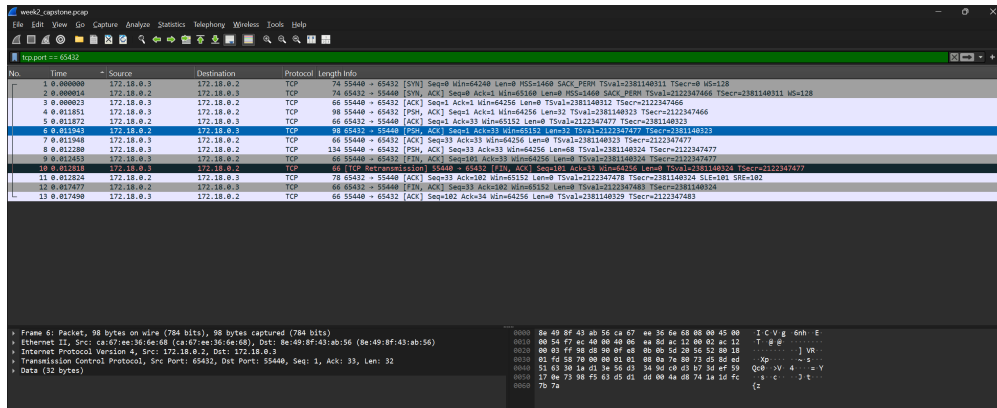


Figure 4: Week 2 Plaintext X25519 Ephemeral Public Key (32 bytes)

Figure 4 shows the 32-byte X25519 public point being transmitted in the clear. This is used to establish the shared session secret before the bulk encryption begins.

4 Defensive Implementation Log

- **Entropy:**

- **Location:** client_week3.py, Line 77
- **Implementation:** nonce = os.urandom(12)
- **Explanation:** The 12-byte (96 bit) nonce required for AES-GCM is generated using the os.urandom() function. This function pulls bytes from the operating system's cryptographically secure pseudo-random number generator (CSPRNG). Using a unique, high-entropy nonce for every encryption operation is critical. The nonce should never be reused because it can allow an attacker to reconstruct the authentication key.

- **Timing Attacks:**

- **Location:** server_week3.py, Line 53
- **Implementation:** alice_identity_pub.verify(...)
- **Explanation:** This implementation utilizes constant-time comparison internally to verify the RSA signature. Standard return or byte comparison (==) is dangerous for signatures. This is because it returns false as soon as it finds the first mismatched byte. An attacker can measure these differences in processing time to guess the correct signature. Constant-time operations ensure the comparison takes the same amount of time regardless of how many bytes match.

- **Fail-Safe Defaults:**

- **Location:** server_week3.py, Lines 53-58 and 89-95
- **Implementation:** This script uses try-except blocks to catch Invalid Signature and Invalid Tag exceptions.
- **Explanation:** When the server encounters an InvalidSignature during the handshake or an InvalidTag during decryption, it will print an error locally and closes the socket. The system does not attempt to request re-transmission, as doing so could open the door for a side-channel attack.

5 AI Research & Standards Validation Log

- **Week 1 (Modes of Operation):**

- **Prompt:** *"Explain the mathematical difference between AES-CBC and AES-GCM. Why does AES-GCM fail catastrophically if a nonce is reused?"*
- **Summary of the AI's output:** *"AES-CBC and AES-GCM differ mainly in structure and guarantees: CBC chains blocks using XOR with the previous ciphertext $C_i = E_k(P_i \oplus C_{i-1})$ and provides only confidentiality, while GCM uses counter mode $C_i = P_i \oplus E_k(\text{nonce}||\text{ctr})$ plus a polynomial authentication function (GHASH over $GF(2^{128})$) to provide both encryption and integrity. The danger in AES-GCM comes from nonce reuse: because counter mode turns AES into a stream cipher, reusing a nonce repeats the keystream, letting attackers recover relationships between plaintexts $C_1 \oplus C_2 = P_1 \oplus P_2$, and more critically, GHASH's linear structure allows attackers to solve for the authentication key and forge valid tags, completely breaking both confidentiality and integrity—hence the failure is catastrophic, unlike CBC where IV reuse only leaks limited information."*
- **Verification:** Based on NIST SP 800-38D, the AI's explanation of Galois field multiplication and nonce constraints was accurate. It correctly explained that CBC IV reuse only leaks limited info, GCM nonce reuse is catastrophic because it breaks the integrity of the entire session. This aligns with the NIST's strict requirement for uniqueness.

- **Week 2 (Key Exchange):**

- **Prompt:** *"Compare the elliptic curves X25519 and NIST P-256. What are the historical security concerns regarding the NIST curve, and why was X25519 developed as an alternative?"*
- **Summary of the AI's output:** *"X25519 (Curve25519) and NIST P-256 are both 256-bit elliptic curves for key exchange, but they differ mainly in trust, design, and ease of implementation. P-256, standardized by National Institute of Standards and Technology, remains secure with no known practical breaks, but its parameter generation lacked transparency and came under suspicion after the Edward Snowden leaks revealed an NSA-influenced backdoor in another NIST standard (Dual_EC_DRBG), raising concerns about possible hidden weaknesses. In contrast, X25519, designed by Daniel J. Bernstein, was created as a transparent, "nothing-up-my-sleeve" alternative with simpler, faster, and more robust implementations that reduce the risk of developer errors and side-channel attacks, which is why it is now preferred in modern protocols."*
- **Verification:** Based on RFC 7748, the AI correctly described the mathematical differences between the curve types. The concerns regarding the NIST curves are valid engineering considerations because X25519's "nothing up my sleeve" parameters were created due to the lack of transparency in the P-256 generation process.

- **Week 3 (Side Channels):**

- **Prompt:** *"Write a Python script demonstrating a timing attack against a standard string comparison (==) used for verifying a MAC or Signature. Explain how the $O(n)$ early-exit leak allows an attacker to guess the value character-by-character."*
- **Summary of the AI's output:**

```
1 import time
2 import string
3 import random
4
5 # --- Victim code (vulnerable) ---
```

```

6 SECRET = "supersecretkey"
7
8 def insecure_compare(a, b):
9     """Early-exit comparison (simulates == behavior)."""
10    if len(a) != len(b):
11        return False
12    for x, y in zip(a, b):
13        if x != y:
14            return False
15        # Artificial delay to amplify timing differences
16        time.sleep(0.005)
17    return True
18
19 def verify_mac(user_input):
20     return insecure_compare(user_input, SECRET)
21
22
23 # --- Attacker code ---
24 def measure_time(guess, trials=3):
25     """Measure average response time for a guess."""
26     total = 0
27     for _ in range(trials):
28         start = time.perf_counter()
29         verify_mac(guess)
30         total += (time.perf_counter() - start)
31     return total / trials
32
33
34 def timing_attack(length):
35     alphabet = string.ascii_lowercase
36     recovered = ""
37
38     for position in range(length):
39         timings = []
40
41         for ch in alphabet:
42             guess = recovered + ch + "a" * (length - len(recovered) - 1)
43             t = measure_time(guess)
44             timings.append((t, ch))
45
46         # Choose character with longest average time
47         timings.sort(reverse=True)
48         best_char = timings[0][1]
49         recovered += best_char
50
51         print(f"Recovered so far: {recovered}")
52
53     return recovered
54
55
56 if __name__ == "__main__":
57     guessed = timing_attack(len(SECRET))
58     print(f"\nActual secret: {SECRET}")
59     print(f"Recovered guess: {guessed}")

```

"A timing attack exploits the fact that native string comparisons (like `==`) often check characters one by one and stop at the first mismatch, making their runtime proportional to how many leading characters are correct. This creates an $O(n)$ side-channel leak: guesses with more correct prefix characters take slightly longer to process. An attacker can repeatedly measure response times for different inputs, identify which guesses take longer, and reconstruct the secret (such as a MAC or signature) character-by-character. The vulnerability arises from early-exit logic and observable timing differences, and it can be mitigated by using constant-time comparison functions (e.g., `hmac.compare_digest` in Python) that always process the entire input without leaking partial information."

- **Verification:** The AI's script accurately represents a timing vulnerability. Python's `hmac.compare_digest()` thwarts this by operating in $O(1)$ constant time. It compares every byte of the inputs regardless of mismatch. This ensures the total execution time is consistent and does not leak the length of the correct prefix to an attacker.

A Client Final Code (client.py)

```

1 import socket
2 import os
3 from cryptography.hazmat.primitives import serialization, hashes
4 from cryptography.hazmat.primitives.asymmetric import rsa, padding
5 from cryptography.hazmat.primitives.asymmetric import x25519
6 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
7 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
8
9 def load_keys():
10     """Loads Alice's private identity key and Bob's public identity key."""
11     with open("alice_private.pem", "rb") as f:
12         alice_priv = serialization.load_pem_private_key(f.read(), password=None)
13     with open("bob_public.pem", "rb") as f:
14         bob_pub = serialization.load_pem_public_key(f.read())
15     return alice_priv, bob_pub
16
17 def start_client():
18     host, port = 'bob_node', 65432
19     message = b"CLASSIFIED: The Eagle flies at midnight."
20     alice_identity_priv, bob_identity_pub = load_keys()
21
22     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
23         try:
24             s.connect((host, port))
25             print(f"[*] Connected to Bob at {host}:{port}")
26
27             # --- PHASE 1: AUTHENTICATED ECDH HANDSHAKE ---
28             # 1. Generate Alice's ephemeral X25519PrivateKey
29             alice_ephemeral_priv = x25519.X25519PrivateKey.generate()
30
31             # 2. Extract Alice's X25519 public bytes
32             alice_pub_bytes = alice_ephemeral_priv.public_key().public_bytes(
33                 encoding=serialization.Encoding.Raw,
34                 format=serialization.PublicFormat.Raw
35             )
36
37             # 3. Use alice_identity_priv to SIGN the X25519 public bytes using padding.PSS
38             # and hashes.SHA256()
39             alice_signature = alice_identity_priv.sign(
40                 alice_pub_bytes,
41                 padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
42                     MAX_LENGTH),
43                 hashes.SHA256()
44             )
45             # 4. Send Alice's signature (256 bytes) and Alice's X25519 public key (32 bytes)
46             # to Bob.
47             s.sendall(alice_signature + alice_pub_bytes)
48
49             # 5. Receive Bob's data (Format: 256-byte signature + 32-byte X25519 public key)
50             data = s.recv(288)
51             bob_sig = data[:256]
52             bob_pub_bytes = data[256:]
53
54             # 6. Use bob_identity_pub to VERIFY Bob's signature against his X25519 public
55             # bytes.
56             bob_identity_pub.verify(
57                 bob_sig,
58                 bob_pub_bytes,
59                 padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
60                     MAX_LENGTH),
61                 hashes.SHA256()
62             )
63             print("[+] Bob's signature verified.")
64
65             # 7. Compute the shared_secret using the verified X25519 points.
66             bob_ephemeral_pub = x25519.X25519PublicKey.from_public_bytes(bob_pub_bytes)
67             shared_secret = alice_ephemeral_priv.exchange(bob_ephemeral_pub)
68
69             # --- PHASE 2: KEY DERIVATION ---
70             # Derive the AES key using HKDF (Same as Week 2)
71             derived_key = HKDF(
72                 algorithm=hashes.SHA256(),
73                 length=32,
74                 salt=None,
75                 info=b'handshake data'

```

```

72         ).derive(shared_secret)
73
74         # --- PHASE 3: SECURE PAYLOAD ---
75         # Encrypt and send the AES-GCM payload (Same as Week 1 & 2)
76         # Extract nonce, tag, and ciphertext.
77         nonce = os.urandom(12)
78
79         cipher = Cipher(algorithms.AES(derived_key), modes.GCM(nonce))
80         encryptor = cipher.encryptor()
81         ciphertext = encryptor.update(message) + encryptor.finalize()
82         tag = encryptor.tag
83
84         s.sendall(nonce + tag + ciphertext)
85         print("[*] Handshake complete. Encrypted payload sent.")
86
87     except ConnectionRefusedError:
88         print("[-] Connection refused. Is Bob's server running?")
89
90 if __name__ == "__main__":
91     start_client()

```

B Server Final Code (server.py)

```

1 import socket
2 from cryptography.hazmat.primitives import serialization, hashes
3 from cryptography.hazmat.primitives.asymmetric import rsa, padding
4 from cryptography.hazmat.primitives.asymmetric import x25519
5 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
6 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
7 from cryptography.exceptions import InvalidTag, InvalidSignature
8
9 def load_keys():
10     """Loads Bob's private identity key and Alice's public identity key."""
11     with open("bob_private.pem", "rb") as f:
12         bob_priv = serialization.load_pem_private_key(f.read(), password=None)
13     with open("alice_public.pem", "rb") as f:
14         alice_pub = serialization.load_pem_public_key(f.read())
15     return bob_priv, alice_pub
16
17 def start_server():
18     host, port = '0.0.0.0', 65432
19     bob_identity_priv, alice_identity_pub = load_keys()
20
21     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
22         s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
23         s.bind((host, port))
24         s.listen()
25         print(f"[*] Bob (Server) listening on {host}:{port}...")
26
27     conn, addr = s.accept()
28     with conn:
29         # --- PHASE 1: AUTHENTICATED ECDH HANDSHAKE ---
30         # 1. Generate Bob's ephemeral X25519PrivateKey
31         bob_ephemeral_priv = x25519.X25519PrivateKey.generate()
32
33         # 2. Extract Bob's X25519 public bytes
34         bob_pub_bytes = bob_ephemeral_priv.public_key().public_bytes(
35             encoding=serialization.Encoding.Raw,
36             format=serialization.PublicFormat.Raw
37         )
38
39         # 3. Use bob_identity_priv to SIGN the X25519 public bytes using padding.PSS and
40         hashes.SHA256()
41         bob_signature = bob_identity_priv.sign(
42             bob_pub_bytes,
43             padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
44             MAX_LENGTH),
45             hashes.SHA256()
46         )
47
48         # 4. Receive Alice's data (Format: 256-byte signature + 32-byte X25519 public
49         key)
50         data = conn.recv(288)
51         alice_sig = data[:256]
52         alice_pub_bytes = data[256:]

```

```

50
51     # 5. Use alice_identity_pub to VERIFY Alice's signature against her X25519
public bytes.
52     #         (If verification fails, cryptography will raise an InvalidSignature
exception. Let it crash the connection.)
53     alice_identity_pub.verify(
54         alice_sig,
55         alice_pub_bytes,
56         padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
MAX_LENGTH),
57         hashes.SHA256()
58     )
59     print("[+] Alice's signature verified.")
60
61     print("[*] Handshake complete. Waiting for payload...")
62
63     # 6. Send Bob's signature and Bob's X25519 public bytes to Alice.
64     conn.sendall(bob_signature + bob_pub_bytes)
65
66     # 7. Compute the shared_secret using the verified X25519 points.
67     alice_ephemeral_pub = x25519.X25519PublicKey.from_public_bytes(alice_pub_bytes)
68     shared_secret = bob_ephemeral_priv.exchange(alice_ephemeral_pub)
69
70     # --- PHASE 2: KEY DERIVATION ---
71     # Derive the AES key using HKDF (Same as Week 2)
72     derived_key = HKDF(
73         algorithm=hashes.SHA256(),
74         length=32,
75         salt=None,
76         info=b'handshake data'
77     ).derive(shared_secret)
78
79     # --- PHASE 3: RECEIVE PAYLOAD ---
80     # Receive and decrypt the AES-GCM payload (Same as Week 1 & 2)
81     data = conn.recv(4096)
82     if data:
83         # Extract nonce, tag, and ciphertext.
84         nonce = data[:12]
85         tag = data[12:28]
86         ciphertext = data[28:]
87
88         # Call your Week 1 decrypt_payload() using the NEW derived key.
89         try:
90             cipher = Cipher(algorithms.AES(derived_key), modes.GCM(nonce, tag))
91             decryptor = cipher.decryptor()
92             plaintext = decryptor.update(ciphertext) + decryptor.finalize()
93             print(f"[+] Decrypted Message: {plaintext.decode()}")
94         except InvalidTag:
95             print("[-] Decryption failed: Invalid tag or key mismatch.")
96
97 if __name__ == "__main__":
98     start_server()

```

C Video Presentation

[Video Presentation Link](#)